

Discussion 2: The Specialists Inside, and How a Word Comes Out

Specialized Heads, and From Signals to Tokens

Gamut Technology Services · AI Engineering Academy · Week 4, Module 01

Format: facilitated group discussion · Target duration: 60 minutes

Companion slides: Module 01, slides 10 to 15

In Discussion 1 you built a picture of attention: the model scores which tokens are relevant to which, from several angles at once. Today we go one level deeper on two fronts. First, the heads are not interchangeable. They take on specific jobs. Second, we follow the pipeline all the way to the end and see how the model's internal state becomes a single spoken word.

What you should be able to do by the end

1. Give two examples of what a head can specialize in, such as grammar structure, meaning, position, or copying.
2. Describe an induction head as prefix-match-and-copy, and connect it to why few-shot prompting works at all.
3. Describe a name mover head as retrieving a specific earlier token and moving it forward.
4. Trace the path from the final layer vector to logits, to a probability distribution, to a chosen token.
5. Explain temperature as a determinism knob, and say why the same prompt can produce different answers.

Three fresh observations from Timberline

SETUP: WHAT THE TEAM NOTICED THIS WEEK

- The team put three examples of “SKU: description” into a prompt, then a fourth SKU with no description. The assistant produced a matching, well-formatted description, even though nobody trained it for that specific task.
- In a long chat, the assistant sometimes recalls a product name mentioned much earlier, and sometimes drops it.
- A support engineer complains the assistant “is not deterministic.” The same question gives slightly different wording each run.

By the end of this discussion, you will be able to explain all three.

Segment A. Heads have jobs

Slide 10. Different heads learn to specialize. The deck names four families: syntactic patterns such as subject and verb agreement, semantic relationships such as meaning and similarity, positional

information such as word order, and copying, where a head simply moves a token from one place to another.

This is division of labor, the same idea behind splitting a system into focused services rather than one function that does everything. Each head owns a narrow concern and gets good at it.

DISCUSSION PROMPT A

You have four heads and four jobs to hand out: subject and verb agreement, coreference (what a pronoun refers to), tracking position, and copying a token from earlier. Take this sentence:

“The manager who approved the return said she would follow up.”

- Where does coreference matter in this sentence?
- Where would a copying head be useful?
- Which job feels hardest to learn from data alone, and why?

QUICK CHECK A

Name two different things an attention head can specialize in.

Segment B. Induction heads, or why examples in a prompt work

Slide 11. An induction head recognizes a repeated pattern and completes it. The plain version: if the model has already seen “A then B” earlier in the text, and it now sees “A” again, it predicts “B.” The deck calls this prefix-match-and-copy.

Under the hood it takes two cooperating heads. A previous-token head in an earlier layer tags each position with the token that came before it. An induction head in a later layer uses those tags to find the earlier match and copy what followed. You do not need the wiring to use the idea. What matters is the behavior: match a pattern you have seen, continue it.

This is the mechanism behind in-context learning. It is why you can drop a few examples into a prompt and the model follows the pattern, with no fine-tuning. The Timberline SKU-description trick is exactly this.

DISCUSSION PROMPT B

A teammate says: “Few-shot prompting is basically magic. You give it examples and it just works.”

- Using prefix-match-and-copy, explain in plain terms why giving examples in the prompt actually changes the output.
- What does this predict will happen if your three examples are inconsistent with each other?
- How would you order or format your examples to make the pattern as easy to match as possible?

Natural break point. Take five minutes before Segment C.

Segment C. Name mover heads, or reaching back for a fact

Slide 12. A name mover head retrieves a specific earlier token, often a proper noun or a fact, and moves it forward to where it is needed. When the model has to recall a name mentioned earlier, this kind of head attends back to that token and carries it to the current position.

A programming analogy: it behaves like a variable reference resolving to an earlier assignment. The name was defined earlier in the context, and this head is the lookup that brings the value forward. These heads are what let a model stay consistent about names and details across a long answer.

DISCUSSION PROMPT C

In a long support chat, a customer mentioned “the Cedarline patio heater” about thirty messages ago. Now they ask a follow-up that needs that product name.

- When should the model be able to pull that name back correctly?
- What could cause it to lose the name or grab the wrong one?
- If you were designing the chat feature, what would you do to make correct recall more likely?

Segment D. From an internal vector to a spoken word

Slides 13 to 15. Now the payoff. After the layers finish, each position holds a rich vector. To turn that into a prediction, the model does the following:

6. **Logits.** The final vector is projected onto the whole vocabulary, giving one raw score per possible token. These raw, unbounded scores are called logits. A vocabulary can be very large, so there are many of them at every step.
7. **Softmax.** Those scores are normalized into a probability distribution. The deck's example: raw scores of 3.0, 2.75, 2.08 for cat, dog, bird become roughly 45 percent, 35 percent, 18 percent.
8. **Sampling.** The model picks a token from that distribution. Strategies like top-k, nucleus (top-p), and temperature control how much randomness is allowed.
9. **Token selection.** One token comes out. Then the whole process repeats for the next token.

Temperature is the determinism knob. At temperature 0 the model takes the top choice every time, which is the closest thing to deterministic. Raise the temperature and the distribution flattens, so lower-ranked tokens get picked more often and outputs vary. One honest caveat worth remembering: temperature 0 gives best-effort determinism on a hosted service, not a hard guarantee of byte-identical output across runs, model versions, or infrastructure. So “run it once and eyeball it” is not a real test. That is a big reason we will need proper evaluation later in the week.

DISCUSSION PROMPT D

Two engineers disagree. One wants temperature 0 for a compliance-summary feature so results are “reproducible.” The other wants temperature 0.8 for a marketing-copy generator.

- Who is right, and for which feature? Can they both be right?
- What does temperature 0 actually guarantee, and what does it not guarantee?
- If reproducibility truly matters for compliance text, what else besides temperature would you want to pin down?

QUICK CHECK D

Put these in the correct order: softmax, logits, chosen token, final-layer vector.

Wrap-up and bridge

You can now explain all three Timberline observations. The SKU trick is induction heads doing in-context learning. The name recall is name mover heads reaching back. The wording that changes run to run is sampling. You can also trace the pipeline end to end, from vector to logits to probabilities to a chosen token.

Next discussion: how do we test our claims about which part does what, and why does any of this matter for safety and trust? That is where logit analysis and ablation come in.

Quick check answers

1. **A.** Any two of: syntactic structure (such as subject and verb agreement), semantic meaning (such as similarity), positional information (word order), and copying (moving a token from one position to another).
2. **D.** Final-layer vector, then logits, then softmax, then chosen token.